plain

Progetto di Ottimizzazione su Grafo

Simone Alessandro Casciaro

Febbraio 2026

Indice

1 Introduzione **2**

2 Obiettivi **4**

3 Algoritmi **5**
3.1 Capacity Scaling 5
3.2 Shortest Augmenting Path 6
3.3 Capacity Scaling con Shortest Augmenting Path 6
3.4 Dinic 7
3.5 Tabella riassuntiva 8

4 Risultati **9**
4.1 Analisi di Scalabilità 9
4.2 Analisi delle capacità 10
4.3 Analisi su dataset reali 11

5 Conclusioni **14**

Capitolo 1

Introduzione

Uno dei problemi più importanti di ottimizzazione combinatoria è il problema del massimo flusso (Max Flow). Il problema consiste nel trovare il massimo flusso che può essere mandato da un nodo a un altro in un grafo. Il nodo da cui parte il flusso viene chiamato *source*, o *sorgente* in italiano, mentre quello che riceve il flusso viene chiamato *sink*, o *pozzo* in italiano.

Formalmente, sia $G = (\mathcal{N}, \mathcal{A})$ un grafo orientato in cui $\mathcal{N}$ è l'insieme dei nodi ed $\mathcal{A}$ è l'insieme degli archi tali per cui $|\mathcal{N}| = n$ e $|\mathcal{A}| = m$. Ad ogni coppia di nodi $(u, v) \in \mathcal{N} \times \mathcal{N}$, viene associata una capacità non negativa $c(u, v) \geq 0$. Se $(u, v) \notin \mathcal{A}$, allora $c(u, v) = 0$.

Viene inoltre definito il flusso, una funzione $f : \mathcal{N} \times \mathcal{N} \to \mathbb{R}$ tale per cui a ogni arco viene associato un flusso non negativo e sempre inferiore alla capacità dell'arco:

$$0 \leq f(u, v) \leq c(u, v) \quad \forall (u, v) \in \mathcal{A}$$

Dato un nodo sorgente s e un nodo pozzo t, l'obiettivo del problema è massimizzare il flusso uscente da s ed entrante in t:

$$f^* = \sum_{v \in \mathcal{N}} (f(s, v) - f(v, s))$$

Il problema può essere espresso anche come un problema di *Programmazione Lineare* (PL) con una funzione obiettivo e dei vincoli di conservazione del flusso, definendo le variabili f_{uv} per indicare il flusso $f(u, v)$ e le variabili c_{uv} per indicare la capacità $c(u, v)$:

$$
\begin{aligned}
\text{maximize} \quad & z \\
\text{subject to} \quad & \sum_{(u \in \mathcal{N}:(u,v) \in \mathcal{A})} f_{uv} - \sum_{(u \in \mathcal{N}:(v,u) \in \mathcal{A})} f_{uv} = \begin{cases} -z & \text{se } v = s \\ 0 & \forall j \neq s, t \\ z & \text{se } v = t \end{cases}, \\
& 0 \leq f_{uv} \leq c_{uv} \qquad \forall (u,v) \in \mathcal{A}
\end{aligned}
$$

Essendo un problema di programmazione lineare, esiste anche un problema duale, denominato problema del minimo taglio (Min Cut). Si definiscono le variabili duali:

- y_i: le variabili associate ai vincoli di conservazione del flusso nel problema primale $\forall i \in \mathcal{N}$;
- ω_{uv}: le variabili associate ai vincoli di capacità degli archi $\forall (u,v) \in \mathcal{A}$.

La sua formulazione matematica è la seguente:

$$
\begin{aligned}
\text{minimize} \quad & w = \sum_{(u,v) \in \mathcal{A}} c_{uv} \omega_{uv} \\
\text{subject to} \quad & y_v - y_u + \omega_{uv} \geq 0 \qquad \forall (u,v) \in \mathcal{A}, \\
& y_s - y_t \geq 1, \\
& y_i \text{ libere} \qquad \forall i \in \mathcal{N}, \\
& \omega_{uv} \geq 0 \qquad \forall (u,v) \in \mathcal{A}
\end{aligned}
$$

Grazie al teorema della dualità in forma forte, si ha un'importante risultato che collega il problema primale (Max Flow) al problema duale (Min Cut):

$$
\exists x^*, c^* : \quad z(x^*) = w(c^*).
$$

Dunque, risolvere all'ottimo il problema del massimo flusso o quello del minimo taglio è equivalente.

Capitolo 2

Obiettivi

Il progetto si pone come obiettivo principale l'analisi comparativa approfondita di diverse strategie risolutive per il problema del Massimo Flusso. Lo studio mira a valutare l'efficienza e la scalabilità di cinque distinti algoritmi, presentati nel dettaglio nel Capitolo 3, mettendone in luce i punti di forza e le criticità attraverso un approccio sperimentale.

Per l'obiettivo, sono stati utilizzati dei grafi presi dal dataset [3], come ad esempio BVZ-tsukuba e K2Z-venus. Questi dataset rappresentano istanze reali derivanti da problemi di segmentazione di immagini, nelle quali ogni pixel è modellato come un nodo del grafo.

Sono anche stati generati dei grafi attraverso una funzione di generazione casuale che, presi in input il numero di nodi n e la densità d, genera un grafo con n nodi e $m_{\max} \cdot d$ archi, dove $m_{\max} = n \cdot (n-1)$ in quanto G è definito come un grafo diretto.

Capitolo 3

Algoritmi

Gli algoritmi scelti per l'analisi sono i seguenti:

- Capacity Scaling;
- Shortest Augmenting Path;
- Capacity Scaling con Shortest Augmenting Path;
- Dinic (versione iterativa);
- Dinic (versione ricorsiva).

3.1 Capacity Scaling

L'algoritmo Capacity Scaling è un miglioramento dell'algoritmo di Ford-Fulkerson [2] che esamina il grafo G partendo dagli archi $(u, v) \in \mathcal{A}$ tali per cui $c(u, v) - f(u, v) \geq \Delta$, dove Δ è un valore progressivamente decrescente che viene inizializzato alla più grande potenza di 2 che sia minore della capacità massima tra gli archi. In questo modo, l'algoritmo cerca di trovare i percorsi dove la capacità residua degli archi è la più alta possibile, lasciando per ultimi quelli che influiscono in maniera minore sul valore del flusso massimo.

Le operazioni che vengono eseguite fintanto che Δ rimane invariato sono racchiuse in una fase chiamata *scaling phase*. Nell'algoritmo Capacity Scaling, il numero di scaling phases è logaritmico, in quanto alla fine di ogni fase il valore di Δ viene diviso per 2 e l'algoritmo termina dopo che la fase $\Delta = 1$

si conclude. Una scaling phase termina quando, per un fissato valore di Δ, non sono presenti cammini aumentanti tali per cui $c(u,v) - f(u,v) \geq \Delta$.

Definendo C la capacità massima tra gli archi del grafo, la complessità temporale dell'algoritmo è $O(m^2 \log C)$. La complessità è dunque polinomiale perché m e $\log C$ sono polinomiali rispetto al numero di bit usati per descrivere l'istanza del problema.

3.2 Shortest Augmenting Path

Lo Shortest Augmenting Path è un altro algoritmo che migliora la base dettata da Ford e Fulkerson. Esso esamina il grafo G partendo dai percorsi più brevi possibile, ovverosia i percorsi con il minor numero di archi attraversati per arrivare da *source* a *sink*.

Si definisce una funzione di distanza sui nodi $d : \mathcal{N} \to \mathbb{N}$ tale per cui:

- $d(t) = 0$;
- $d(u) \leq d(v) + 1 \quad \forall (u,v) \in \mathcal{A} : c(u,v) - f(u,v) > 0$.

All'inizio dell'esecuzione dell'algoritmo, viene eseguita una BFS inversa a partire dal nodo pozzo, etichettando tutti i nodi del grafo con la distanza che essi hanno dal pozzo.

Durante l'esecuzione dell'algoritmo, la ricerca dei cammini aumentanti viene eseguita trattando come *ammissibili* solo gli archi $(u,v) \in \mathcal{A}$ che rispettano le seguenti proprietà:

- $c(u,v) - f(u,v) > 0$;
- $d(u) = d(v) + 1$.

In questo modo, l'algoritmo dà priorità ai percorsi che sono più veloci per arrivare al nodo pozzo, effettuando per ultimi gli aumenti che riguardano percorsi con un numero elevato di archi da percorrere.

La complessità temporale dell'algoritmo è $O(n^2 m)$, rendendolo quindi strettamente polinomiale rispetto al numero di nodi e archi del grafo.

3.3 Capacity Scaling con Shortest Augmenting Path

Capacity Scaling con Shortest Augmenting Path è un algoritmo che combina i due presentati nelle sezioni 3.1 e 3.2. L'idea alla base dell'algoritmo è di

utilizzare Shortest Augmenting Path come una *sub-routine*, limitando la ricerca dei percorsi a quelli che hanno archi con una capacità superiore o uguale a Δ. La logica del calcolo del Δ è uguale a quella di Capacity Scaling: l'obiettivo dell'algoritmo è dunque focalizzarsi su percorsi che hanno pochi archi e con capacità alte, combinando le tecniche dei due algoritmi.

La complessità temporale dell'algoritmo è $O(nm \log C)$, che risulta essere un bilanciamento tra la complessità vista con Shortest Augmenting Path e quella vista con Capacity Scaling.

3.4 Dinic

L'algoritmo di Dinic [1] è un algoritmo fortemente polinomiale inventato nel 1970 per il problema del massimo flusso. L'idea alla base dell'algoritmo è etichettare tutti i nodi con la distanza che hanno dal nodo sink, costruendo il *grafo dei livelli* attraverso una BFS, in maniera analoga alla funzione distanza citato in Shortest Augmenting Path. Una volta costruito il grafo dei livelli, si utilizza la DFS per cercare un cammino aumentante. Quando la DFS non trova un percorso aumentante, viene interrotto il ciclo interno e generato un nuovo grafo dei livelli attraverso la BFS.

In questo progetto vengono presentate due versioni di Dinic, che differiscono nel modo in cui la DFS viene implementata:

- versione iterativa;
- versione ricorsiva.

In entrambi i casi, una volta costruito il grafo dei livelli e assegnato un livello ad ogni nodo del grafo, la DFS ricerca il percorso aumentante usando come archi ammissibili solo gli archi $(u, v) \in \mathcal{A}$ tali per cui:

$$level[v] = level[u] + 1$$

e che hanno ancora flusso disponibile (cioè tali per cui $f(u, v) < c(u, v)$).

La complessità temporale dell'algoritmo è $O(n^2 m)$.

3.5 Tabella riassuntiva

In tabella 3.1 sono presenti le complessità temporali e spaziali degli algoritmi.

Tabella 3.1: Riassunto delle complessità temporali e spaziali

Algoritmo	Complessità Temporale	Complessità Spaziale
Capacity Scaling	$O(m^2 \log C)$	$O(n+m)$
Shortest Augmenting Path	$O(n^2 m)$	$O(n+m)$
CS + SAP	$O(nm \log C)$	$O(n+m)$
Dinic	$O(n^2 m)$	$O(n+m)$

Capitolo 4

Risultati

Per eseguire l'esperimento, è stata utilizzata una macchina dotata di 16GB di RAM e un processore Intel® Core™ i7-11700F con 8 Core. I tempi assoluti presenti nelle analisi di questo report dipendono strettamente dall'hardware utilizzato, dunque l'attenzione verrà posta solo ai tempi relativi e ai confronti tra i vari algoritmi.

4.1 Analisi di Scalabilità

Una delle analisi importanti per degli algoritmi di massimo flusso è la loro *scalabilità*, ovvero l'aumento del tempo di esecuzione all'aumentare della grandezza dell'input.

Come evidenziato in Figura 4.1, l'andamento lineare dei punti nel grafico di dispersione in scala logaritmica indica che i tempi di esecuzione crescono in maniera polinomiale rispetto al numero di archi m. Questo comportamento è coerente con la complessità teorica degli algoritmi analizzati (che dipendono da m e dal numero di nodi n).

Sempre dalla figura 4.1, si nota come l'implementazione ricorsiva di Dinic e Shortest Augmenting Path siano dominanti rispetto ai restanti tre algoritmi. Dinic ricorsivo risulta essere più efficiente quando il numero di archi è ridotto, mentre Shortest Augmenting Path risulta migliore quando il numero di archi è maggiore.

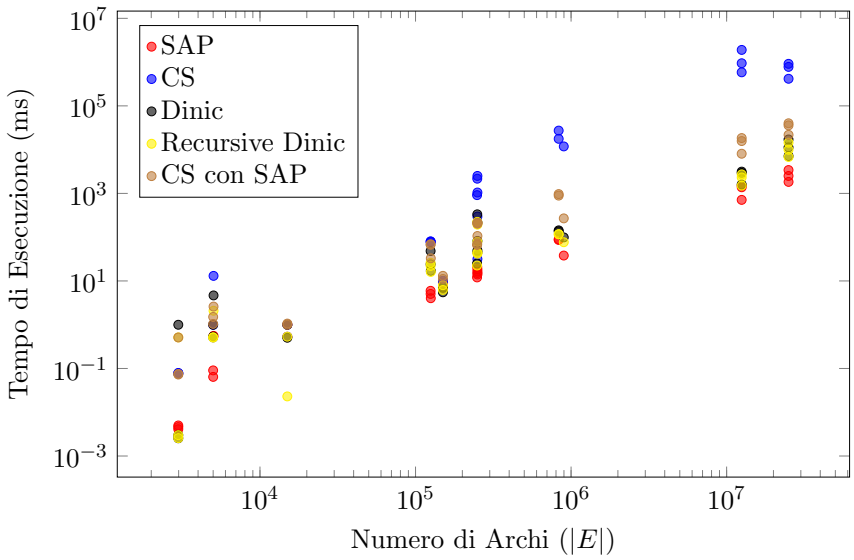

Figura 4.1: Scalabilità

4.2 Analisi delle capacità

Nel grafico di Figura 4.2 viene mostrata la sensibilità degli algoritmi a un cambiamento radicale della *capacità massima*. Usando dei grafi generati casualmente, tutti di dimensione tale per cui $n = 1000$ e

$$m = n \cdot (n - 1) \cdot 0.005 = 4995,$$

sono stati divisi gli input in:

- grafi che hanno capacità massima $C = 10$;
- grafi che hanno capacità massima $C = 100000$.

Dalla teoria, gli unici algoritmi che dovrebbero essere influenzati dalla capacità massima sono Capacity Scaling e Capacity Scaling + Shortest Augmenting Path, che hanno una complessità temporale che dipende da $\log C$.

I risultati visibili in Figura 4.2 confermano questo fatto, con Shortest Augmenting Path che non viene influenzato dal cambiamento, mentre Capacity Scaling presenta tempi circa 12x superiore. Usando Shortest Augmenting Path come subroutine per Capacity Scaling, il tempo migliora e il fattore di peggioramento è di 3x.

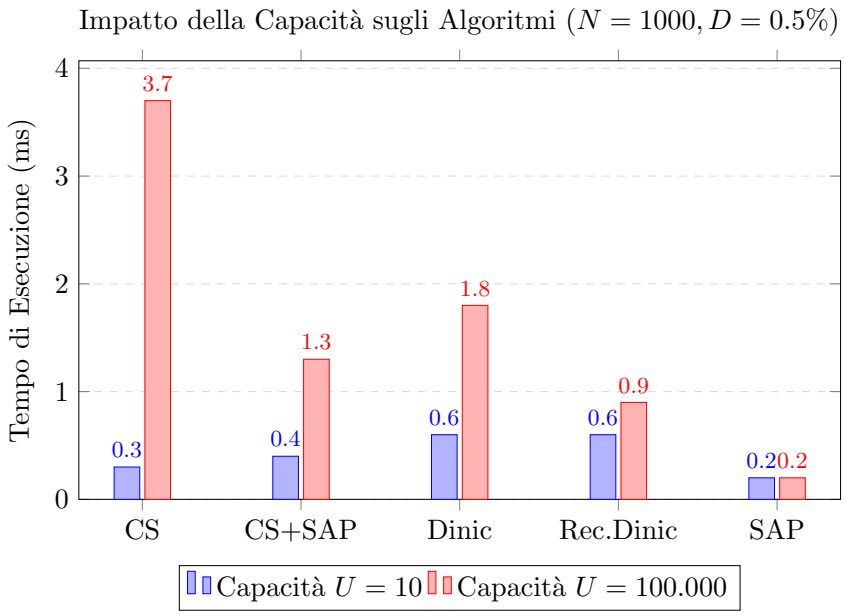

Figura 4.2: Analisi delle Capacità

4.3 Analisi su dataset reali

Dai grafici in Figura 4.3 e 4.4, si può notare come nei grafi di visione artificiale, come BVZ-Tsukuba e K2Z-Venus [3], a dominare sia l'algoritmo di Dinic, nella versione ricorsiva. Shortest Augmenting Path, sia da solo sia nella versione con Capacity Scaling, ha delle prestazioni leggermente inferiori.

Analizzando i risultati ottenuti sul grafo Tsukuba, l'algoritmo più in difficoltà risulta essere nuovamente Capacity Scaling, che risolve il problema in quasi $100s$ (per confronto, Dinic in versione ricorsiva ha risolto il problema in $\approx 0.1s$). L'enorme differenza tra i due algoritmi potrebbe essere data dalla tipologia di grafo utilizzato: BVZ-Tsukuba utilizza capacità relativamente piccole, con $C = 10$, iterando dunque su poche fasi ma molto complesse.

I risultati sul grafo Venus confermano l'analisi e, anzi, amplificano le differenze: Capacity Scaling ha un tempo di esecuzione sull'ordine dei minuti, mentre Shortest Augmenting Path e Dinic ricorsivo si confermano sotto al secondo.

È interessante notare come le differenze prestazionali che riguardano Dinic:

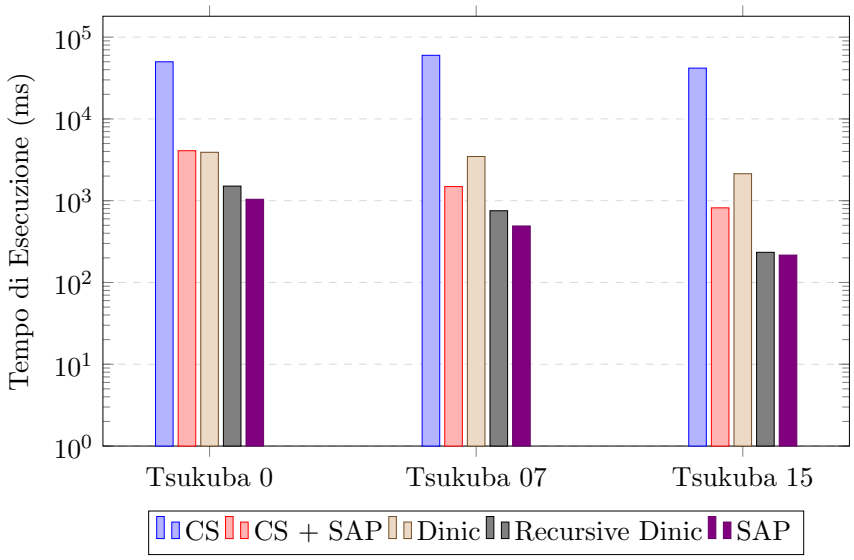

Figura 4.3: Prestazioni sul dataset di Visione Artificiale BVZ-Tsukuba

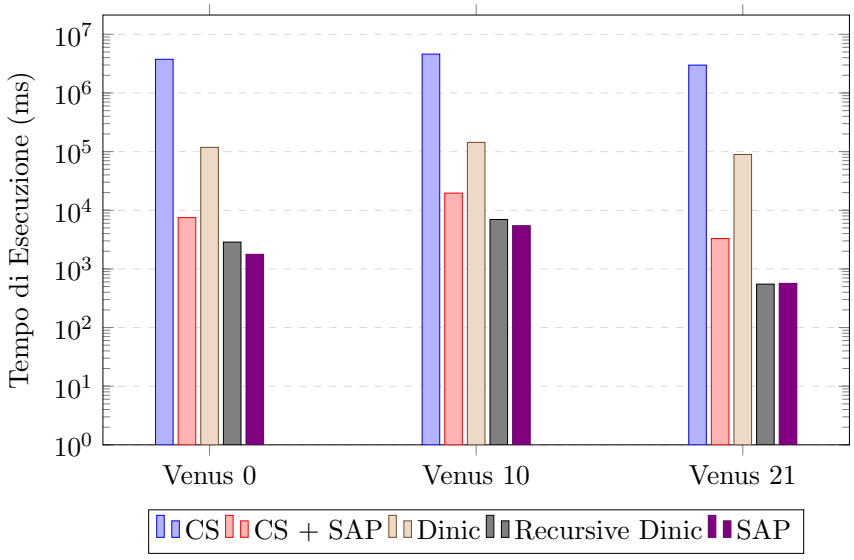

Figura 4.4: Prestazioni sul dataset di Visione Artificiale K2Z-venus

la versione iterativa dell'algoritmo risulta essere decisamente più lenta della versione ricorsiva. Nonostante la logica dell'algoritmo rimanga invariata, le differenze prestazionali potrebbero derivare dalla diversa gestione dello *stack* (dichiarato esplicitamente nella versione iterativa, usato naturalmente mediante lo stack della memoria nella versione ricorsiva).

Capitolo 5

Conclusioni

Negli algoritmi di ottimizzazione, l'analisi sulla complessità temporale teorica, ragionando sul caso peggiore, spesso risulta essere molto pessimistica. In questo progetto, si è voluto effettuare un confronto diretto tra i 5 algoritmi proposti attraverso un approcio empirico.

Gli algoritmi che sono risultati più efficienti sono Shortest Augmenting Path e Dinic ricorsivo, mentre Capacity Scaling risulta essere il meno efficiente.

Combinare Shortest Augmenting Path e Capacity Scaling ha migliorato le prestazioni di Capacity Scaling, ma il risultato è rimasto comunque inferiore a quello di Shortest Augmenting Path utilizzato da solo.

Infine, tra i due algoritmi migliori, benché la complessità temporale teorica di entrambi sia $O(n^2 \cdot m)$, Shortest Augmenting Path risulta essere migliore quando il numero di archi m cresce, mentre Dinic lo è quando il numero di archi è minore.

Bibliografia

[1] DINIC E. A. "Algorithm for solution of a problem of maximum flow in networks with power estimation". In: *Soviet Math. Doklady* 11 (1970), pp. 1277–1280. URL: https://cir.nii.ac.jp/crid/1570854175600484992.

[2] L. R. Ford e D. R. Fulkerson. "Maximal Flow Through a Network". In: *Canadian Journal of Mathematics* 8 (1956), pp. 399–404. DOI: 10.4153/CJM-1956-045-5.

[3] Patrick Møller Jensen et al. "Min-Cut/Max-Flow Problem Instances for Benchmarking". In: (mar. 2022). DOI: 10.11583/DTU.17091101.v1. URL: https://data.dtu.dk/articles/dataset/Min-Cut_Max-Flow_Problem_Instances_for_Benchmarking/17091101.